

Du Structured Reinforcement Learning aux jeux Atari

Etude, adaptation et benchmark

Yacine Klikel Arthur Gorin Léo Cheng

Matteo Le Goff Neil Benmouffok Pierre Bressollette

2025–2026

1 Contexte & motivation

2 Structured Reinforcement Learning (SRL)

3 Résultats

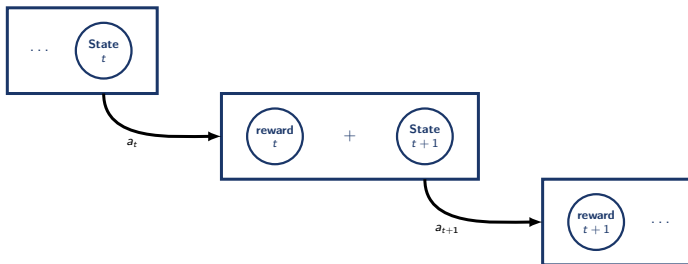
4 Conclusion

5 Annexes

Contexte & motivation

Processus de Décision Markovien (MDP)

- Un ensemble d'états S
- Un ensemble d'actions A
- Une fonction de transition $\mathbb{P}(s' | s, a)$
- Une fonction de récompense $R(s, a)$
- Un facteur d'escompte $\gamma \in [0, 1]$



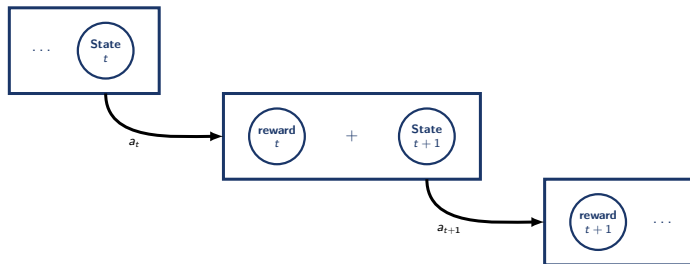
Processus de Décision Markovien (MDP)

- Un ensemble d'états S
- Un ensemble d'actions A
- Une fonction de transition $\mathbb{P}(s' \mid s, a)$
- Une fonction de récompense $R(s, a)$
- Un facteur d'escompte $\gamma \in [0, 1]$



Objectif :

Trouver la politique optimale
 $\pi^*(s) = a$



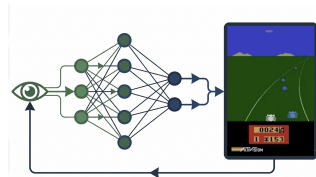
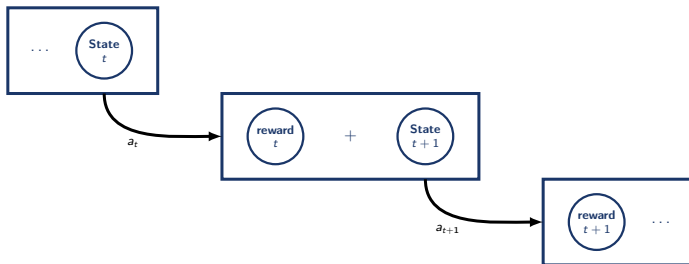
Processus de Décision Markovien (MDP)

- Un ensemble d'états S
- Un ensemble d'actions A
- Une fonction de transition $\mathbb{P}(s' \mid s, a)$
- Une fonction de récompense $R(s, a)$
- Un facteur d'escompte $\gamma \in [0, 1]$



Objectif :

Trouver la politique optimale
 $\pi^*(s) = a$



Cadre : $\text{MDP} = (\mathcal{S}, \mathcal{A}, \mathbb{P}, R, \gamma)$

Estimation de la pertinence d'un couple état/action $Q(s, a)$

- initialement tabulé
- Arrivée du deep learning : approximation de Q par un réseau de neurone Q_θ

Cadre : MDP = $(S, A, \mathbb{P}, R, \gamma)$

Estimation de la pertinence d'un couple état/action $Q(s, a)$

- initialement tabulé
- Arrivée du deep learning : approximation de Q par un réseau de neurone Q_θ

Le blocage en Recherche Opérationnelle (RO) :

- Problèmes réels (*vehicle scheduling*, logistique) = espaces combinatoires géants
- Les réseaux des algorithmes classiques n'arrivent pas à acquérir une représentation utile du problème

Cadre : $\text{MDP} = (\mathcal{S}, \mathcal{A}, \mathbb{P}, R, \gamma)$

Estimation de la pertinence d'un couple état/action $Q(s, a)$

- initialement tabulé
- Arrivée du deep learning : approximation de Q par un réseau de neurone Q_θ

Le blocage en Recherche Opérationnelle (RO) :

- Problèmes réels (*vehicle scheduling*, logistique) = espaces combinatoires géants
- Les réseaux des algorithmes classiques n'arrivent pas à acquérir une représentation utile du problème

Objectif du projet :

- Implémentation de l'algorithme SRL dans un autre cadre d'application : Les jeux Atari

Structured Reinforcement Learning (SRL)

SRL : définition

- **Définition** : Intégration d'une couche combinatoire (non différentiable) au sein d'un réseau de neurones. [1]

- **Exemple** :



(A) Carte initiale

NN
⇒



(B) Carte des coûts

Dijkstra
⇒



(C) Trajectoire optimale

SRL : the pipeline

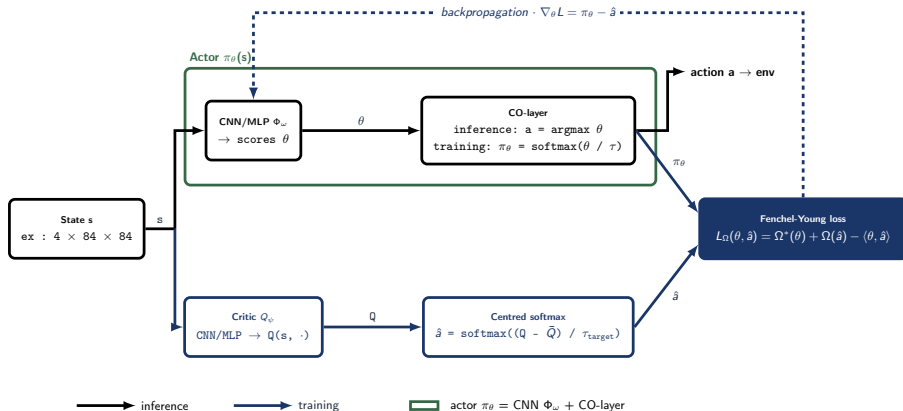
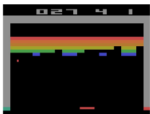


Figure – Architecture du SRL dans le cadre d'un jeu ATARI

SRL : Implémentation

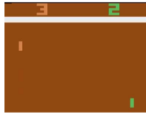
- Six jeux : Breakout, Space Invaders, Q*bert, Beam Rider, Enduro et Pong (*Cas d'étude aboutis : Breakout et Enduro*)



Breakout



Enduro



Pong



Q*bert



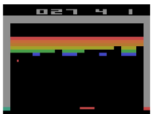
Space invaders



Beam Rider

SRL : Implémentation

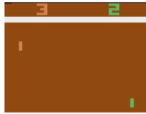
- Six jeux : Breakout, Space Invaders, Q*bert, Beam Rider, Enduro et Pong (*Cas d'étude aboutis : Breakout et Enduro*)



Breakout



Enduro



Pong



Q*bert



Space invaders

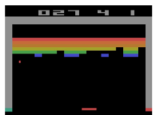


Beam Rider

- Double portage : Julia / Python (*PyTorch + Gymnasium*)

SRL : Implémentation

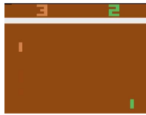
- Six jeux : Breakout, Space Invaders, Q*bert, Beam Rider, Enduro et Pong (*Cas d'étude aboutis : Breakout et Enduro*)



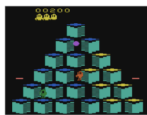
Breakout



Enduro



Pong



Q*bert



Space invaders



Beam Rider

- Double portage : Julia / Python (*PyTorch + Gymnasium*)
- Deux approches différentes :

Approche Pixels / CNN

État : Empilement de 4 frames grises (84×84) normalisées dans $[0, 1]$.

Réseau (ϕ_w) : Convolutionnel (CNN).

Particularité : Voie la plus proche des benchmarks classiques (DQN) [2].

Approche RAM / MLP

État : Vecteur de la mémoire console (RAM Atari) normalisé.

Réseau (ϕ_w) : Perceptron multicouche (MLP).

Particularité : Élimine la charge de la vision (cycle plus court), mais représentation plus opaque.

Résultats

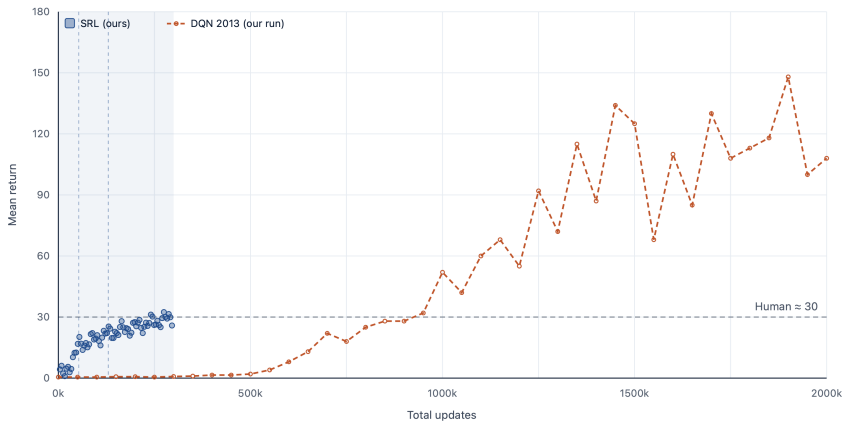
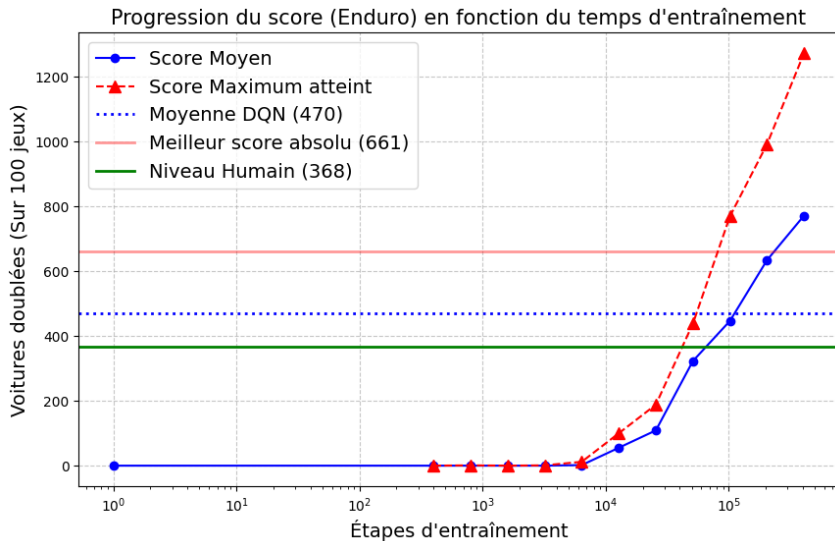


Figure – Comparaison de la vitesse de convergence : SRL (bleu) contre DQN 2013 (rouge, approximatif) sur la même échelle de mises à jour.





Signal d'apprentissage faible

- score stagne à -21 (défaite totale)
- récompenses trop rares
- critic instable
- gradient très faible
- politique bloquée

Analyse

Dans certains jeux, récompenses trop rares
→ apprentissage bloqué dans une politique médiocre.

Conclusion

Apports du SRL

- ⊕ **Efficacité** : Moins d'itérations sur la descente de gradient.
- ⊕ Entraîner un réseau critique en même temps que l'acteur permet d'aboutir à des comportements plus fins qu'un simple DQN
- ⊖ **Sensibilité** : Réglage critique de l'exploration (τ , β , entropie).
- ⊖ **Sous-exploitation** : L'espace d'actions des jeux Atari n'étant pas combinatoire, la CO-layer se réduit ici à un simple *argmax*.

Ouverture : Vers le combinatoire

Justification de la CO-layer

Indispensable dès que l'espace d'actions $|\mathcal{A}|$ devient réellement **combinatoire** :

- Cas où l'apprentissage direct de la vision vers l'action échoue.
- Jeux à structures de graphes complexes (ex : Echecs)
- Passage à l'échelle sur des environnements non-quadrillés.

Merci de votre attention !

Annexes

$$\mathcal{L}_\Omega(\theta; \hat{a}) = \mathbb{E} \left[\max_{a \in \mathcal{A}(s)} (\theta + Z)^\top a \right] - \theta^\top \hat{a} \quad (1)$$

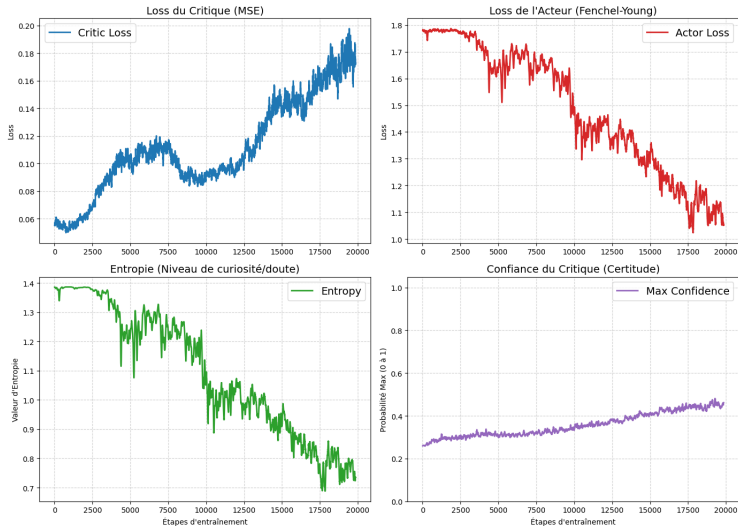
$$\nabla_\theta \mathcal{L}_\Omega(\theta; \hat{a}) = \pi_\theta - \hat{a} = \mathbb{E} \left[\arg \max_{a \in \mathcal{A}(s)} (\theta + Z)^\top a \right] - \hat{a} \quad (2)$$

$$\mathcal{L}_\Omega(\theta; \hat{a}) = \underbrace{\tau \log \sum_a e^{\theta_a / \tau}}_{\Omega^*(\theta)} - \underbrace{\tau \sum_a \hat{a}_a \log \hat{a}_a}_{\Omega(\hat{a})} - \underbrace{\sum_a \theta_a \hat{a}_a}_{\langle \theta, \hat{a} \rangle}, \quad (3)$$

Sous forme régularisée :

$$\nabla_\theta \mathcal{L}_\Omega = \pi_\theta(\cdot \mid s) - \hat{a} \text{ avec } \pi_\theta = \text{softmax}(\theta / \tau) \quad (4)$$

Recherche Hyperparametres



- [1] Heiko HOPPE et al. "Structured Reinforcement Learning for Combinatorial Decision-Making". In : *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. 2026. URL : <https://openreview.net/forum?id=GS9o7u5njS>.
- [2] Volodymyr MNIH et al. "Playing atari with deep reinforcement learning". In : *arXiv preprint arXiv:1312.5602* (2013).